



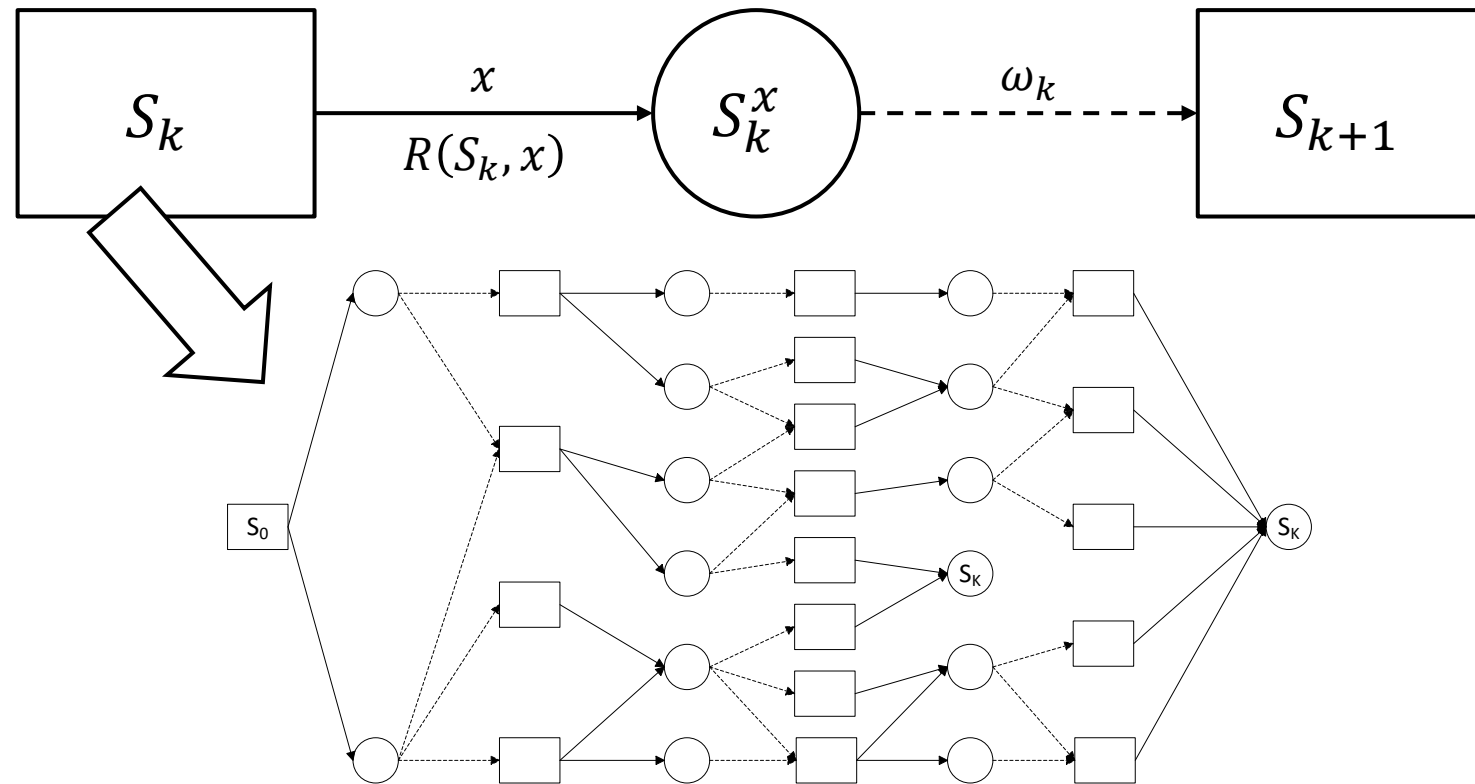
Technische
Universität
Braunschweig



Data Driven Decision Making

Lecture 6 – Approximate Dynamic Programming

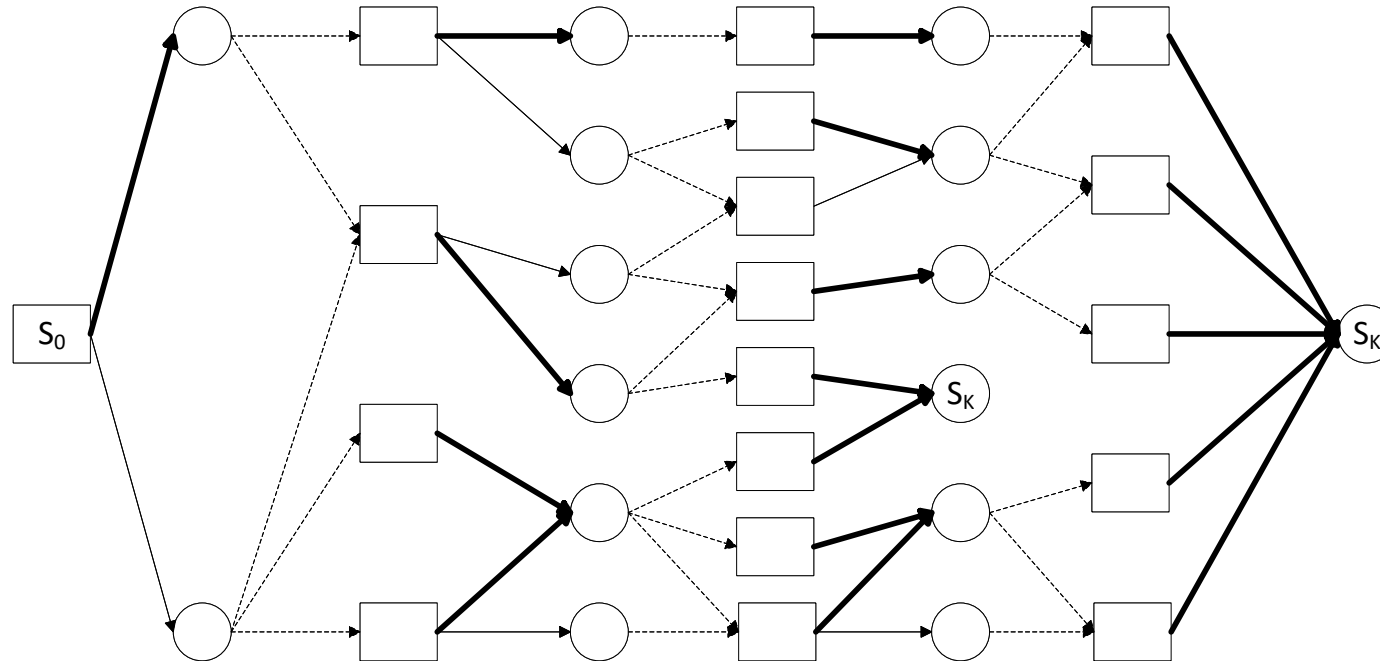
Problem Modeling: Markov Decision Process



- Initial state S_0
- Final state S_K

Solution: Decision Policy

- A decision policy assigns a decision to every state



- Objective: find optimal policy π^* maximizing expected overall rewards

Optimal Solution

- An optimal solution π^* satisfies:

$$\pi^* = \arg \max_{\pi \in \Pi} \mathbb{E} \left[\sum_{i=0}^K R(S_i, X_i^{\pi^*}(S_i)) | S_0 \right]$$

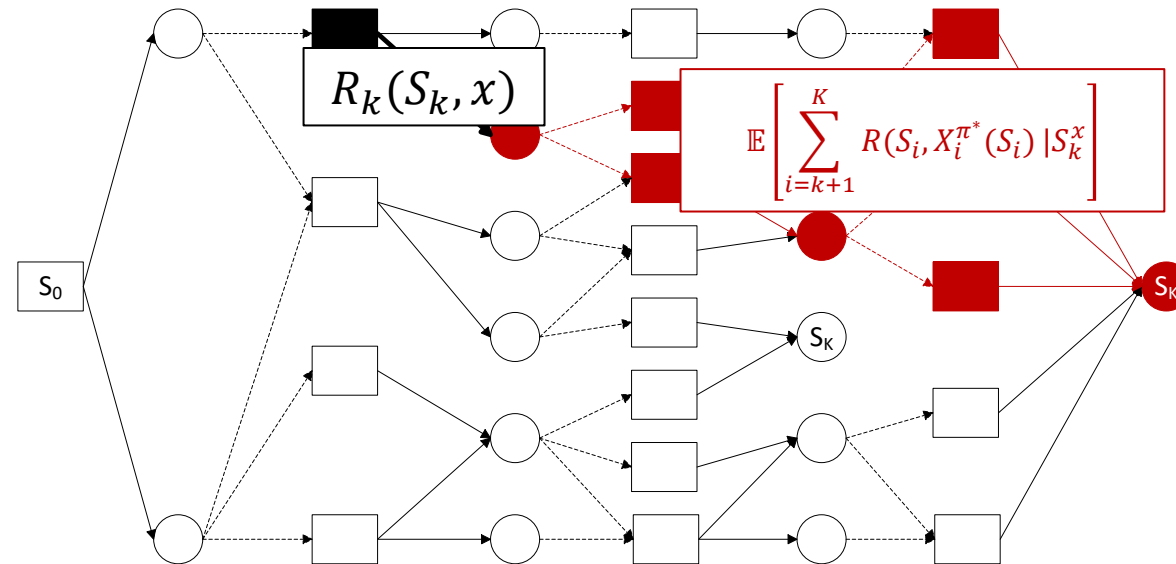
- Policy π^* represents the policy $\pi \in \Pi$, maximizing the expected reward
- Given initial state S_0
- $X_k^{\pi}(S_k)$ indicates the decision made by policy π in state S_k
- For minimization, we search for $\arg \min$

Bellman Equation

- An optimal policy satisfies:

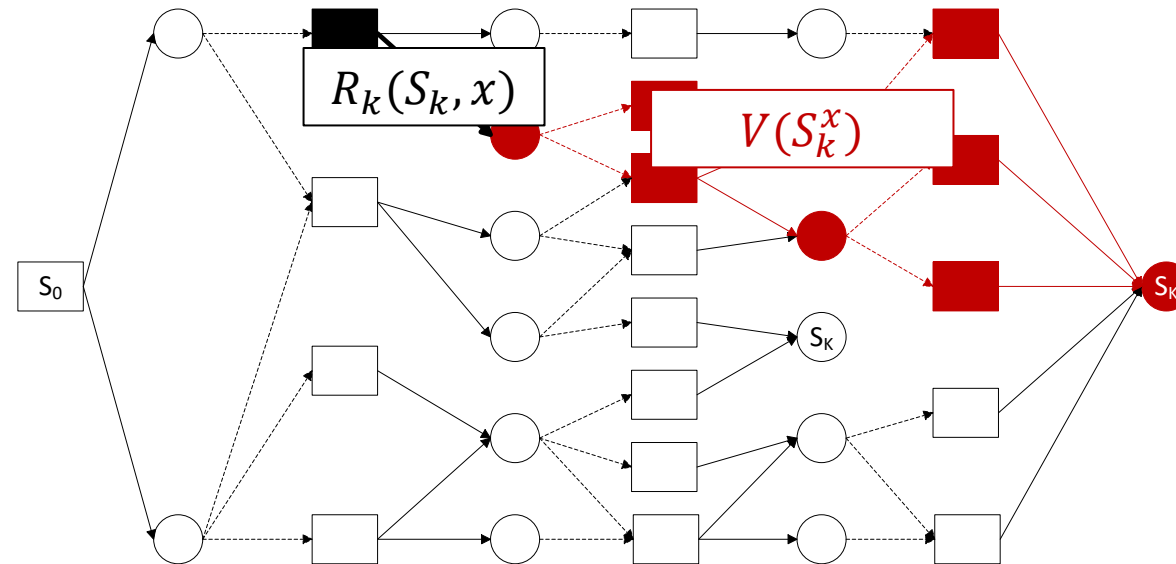
$$X_k^{\pi^*}(S_k) = \arg \max_{x \in X(S_k)} \left(R(S_k, x) + \mathbb{E} \left[\sum_{i=k+1}^K R(S_i, X_i^{\pi^*}(S_i) | S_k^x) \right] \right)$$

- Maximize immediate plus expected future reward



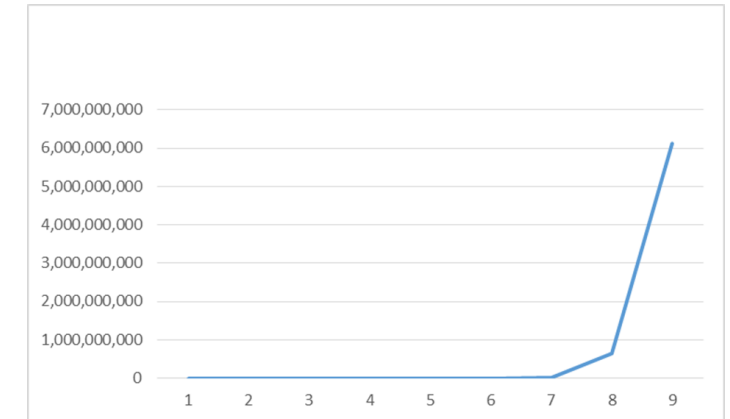
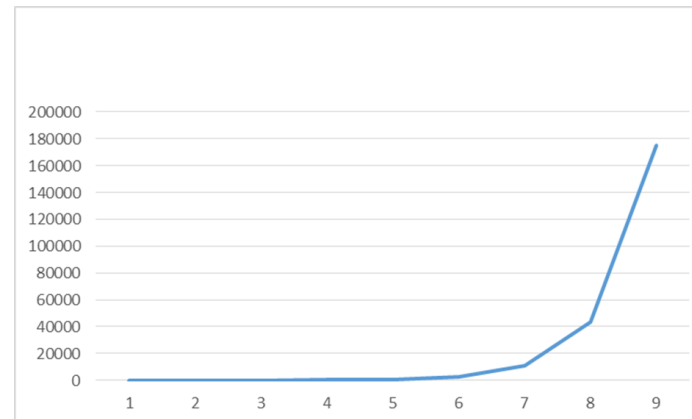
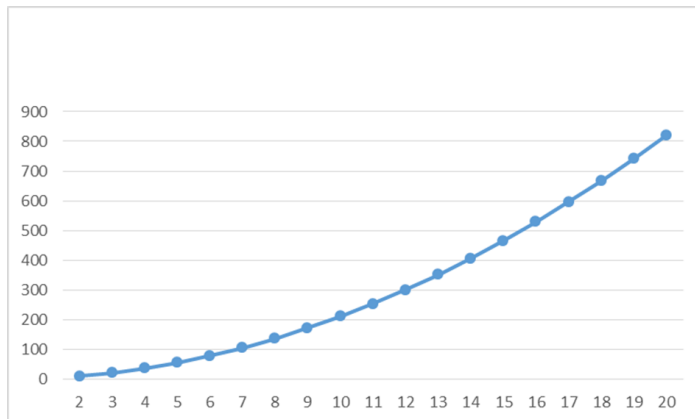
Value Function

- Expected future reward is called the **value** $V(S_k^x)$ of post-decision state S_k^x
- The value function maps a (post-decision) state to the expected future reward



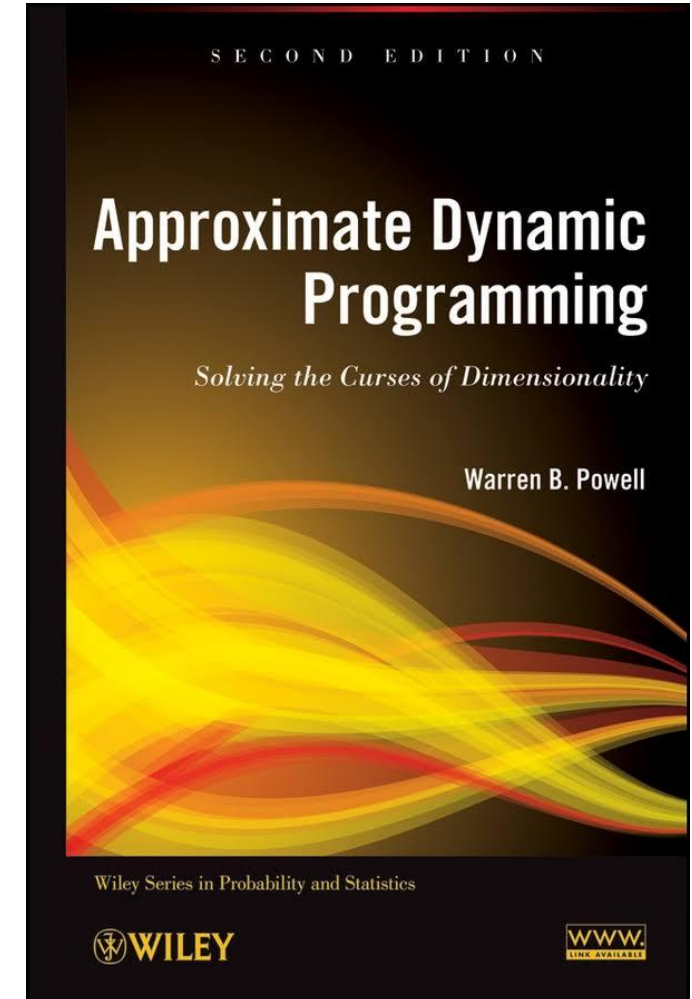
Curses of Dimensionality

- State Space
- Decision Space
- Information Space
- With increasing dimensionality, recursion becomes infeasible.
- For (nearly) all real-world problems, an optimal solution cannot be found



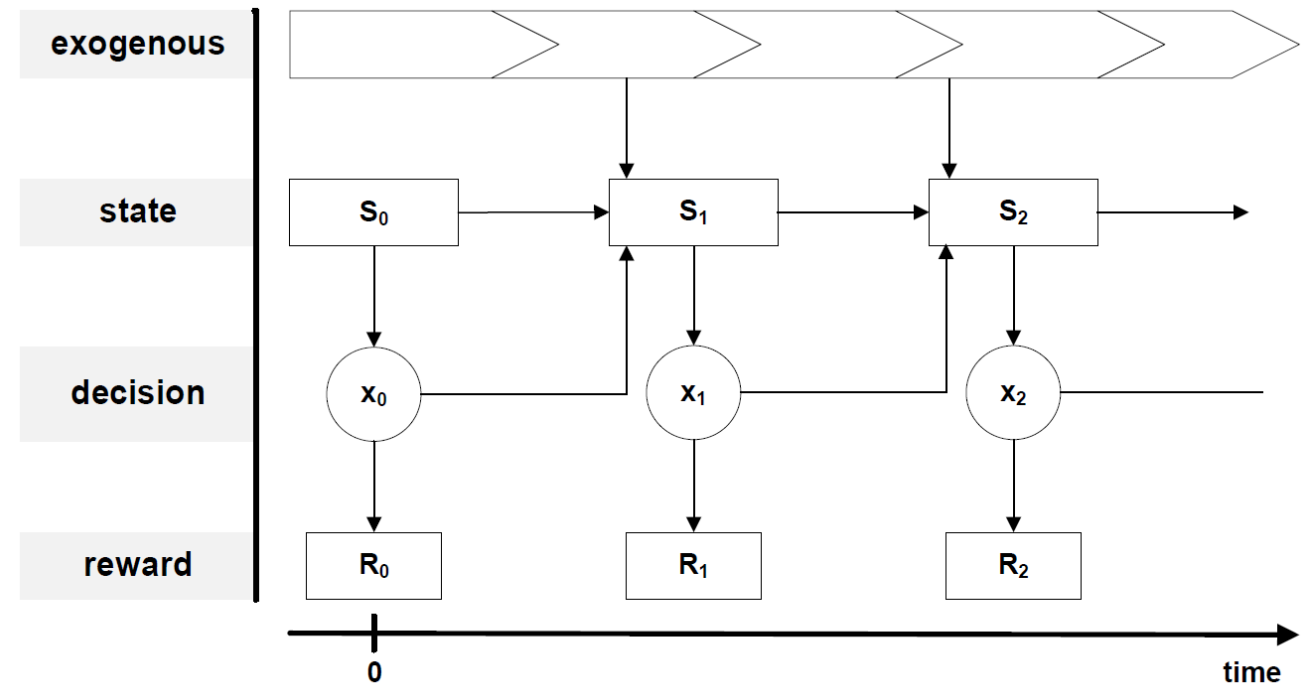
Approximate Dynamic Programming

- Powell 2011: ADP comprises **any way** to come up with a (heuristic) policy
- Problem specific assumptions are engaged in order to generate a good, hopefully near optimal solution.
- In the ADP context we select a “reasonable” decision for every state



Approximate Dynamic Programming

- For example,
 - Focus on high immediate reward or low cost
 - Maintain flexibility of the resources
 - Anticipate potential future developments



Policy Classes (Compare Powell 2011)

- Policy Function Approximation
- Rolling Horizon Re-Optimization
- Cost Function Approximation
- Look-ahead Models
- Value Function Approximation

Rule-Based

Optimization-Based

Agenda

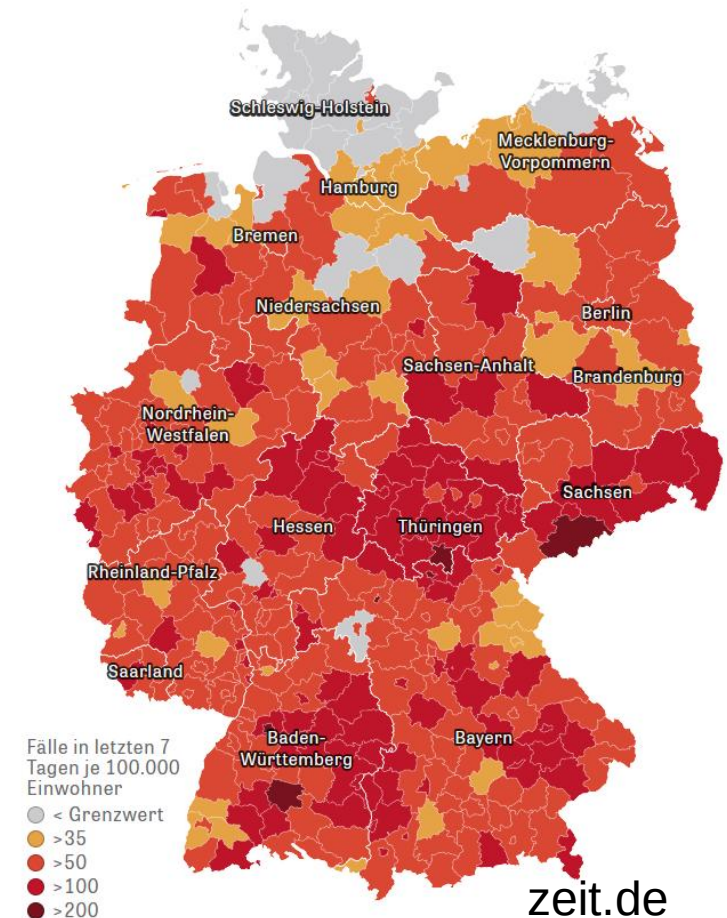
- Policy Function Approximation
- Rolling Horizon Re-Optimization
- Cost Function Approximation

- Case Study: Restaurant Meal Delivery



Policy Function Approximation

- Idea: Transfer rule-of-thumb idea into a policy
- Very problem specific
- Prominent example: threshold-strategies
 - Decisions are based on a measure and a threshold
 - If measure is below threshold, select A, else, select B
- No direct connection with reward- or value-function
- No explicit anticipation
- In some cases: PFA is the result of analytical considerations and proofs



Examples

- Scheduling:
 - FIFO
 - Shortest processing time
 - Longest processing time
 - Earliest due date
- Dynamic Vehicle Routing:
 - Waiting strategies
 - Postponement of assignments
 - Serve customers if detour is less than 5 minutes
- Inventory Management:
 - Always refill to level of 50%
 - Always refill the demand that was observed the day before
- Finance
 - Buy low, sell high
 - Set stop loss at 90%

Summary

- Fast
- Easy implementation, communication, and interpretation.
- Ignores state and decision details.
- Usually no fine tuning. For example, thresholds may depend on state parameters such as time or workload
- 80-20 rule: we get decent results but not high-quality solutions
- Often works even if assumptions about information and decision model are not accurate

Agenda

- Policy Function Approximation
- **Rolling Horizon Re-Optimization**
- Cost Function Approximation

- Case Study: Restaurant Meal Delivery



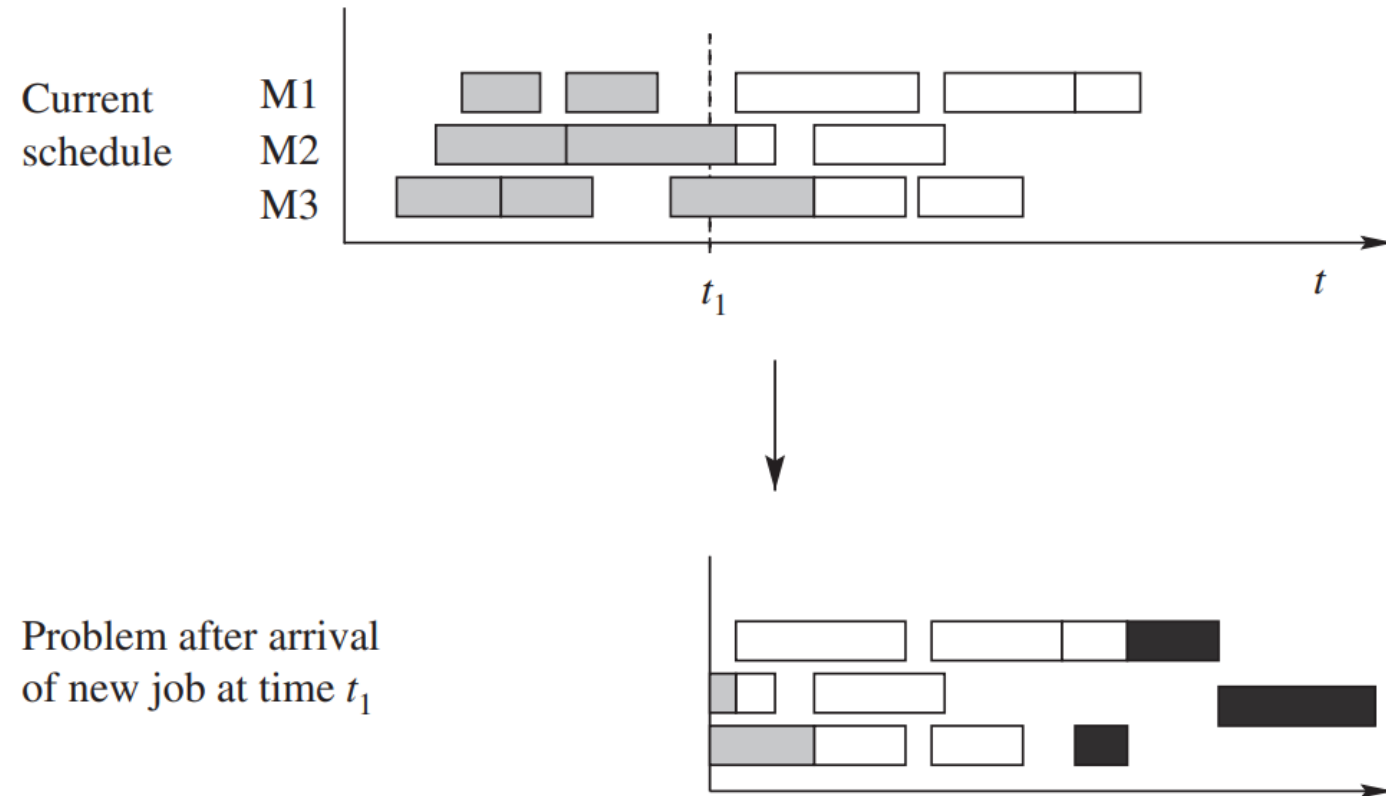
Rolling Horizon Re-Optimization

- Decisions are based on the currently available information
- Maximize immediate reward or minimize immediate costs:
- $x = \operatorname{argmin}_{x \in X(S)} R(S, x)$ ($\rightarrow$ Bellman Equation with $V = 0$)
- Direct application of static mixed-integer programming possible
- Very popular in (early) dynamic decision making literature

Rolling time horizon in scheduling

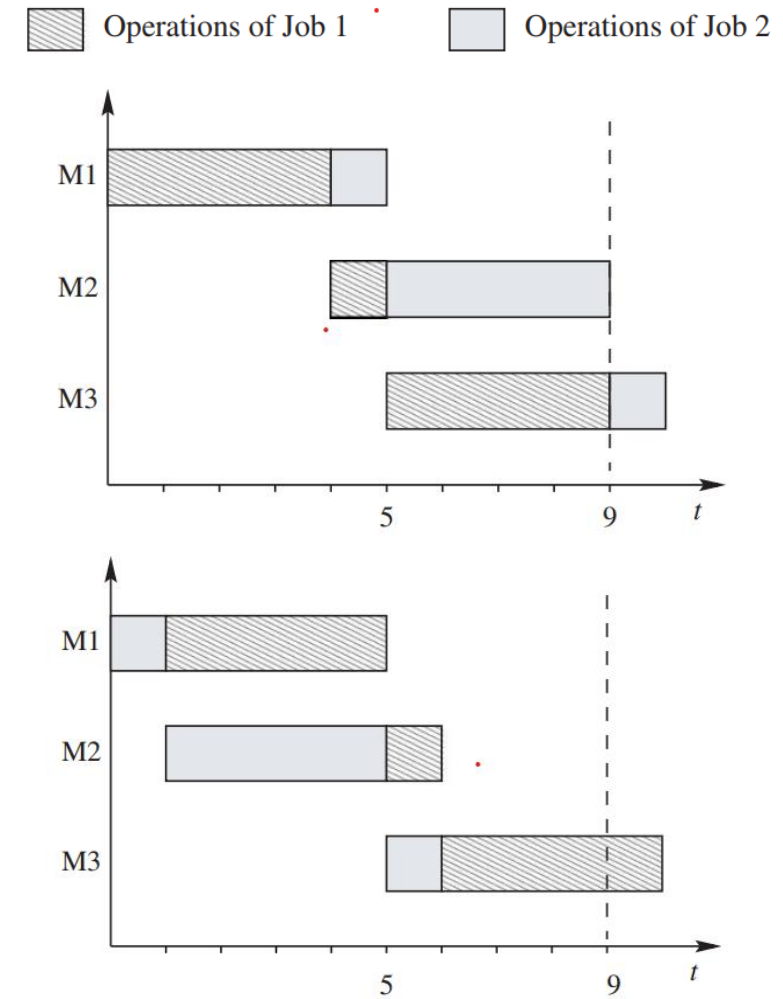
- Anticipation and flexibility in dynamic scheduling, J Branke, DC Mattfeld - International Journal of Production Research, 2005
- At time t_1 ,
 - Remove the jobs already processed
 - Block machines for jobs in processing
 - Add newly released jobs
 - Come up with a new schedule

- Operations fixed at time t_1
- Operations transferred to new problem
- Operations of new job



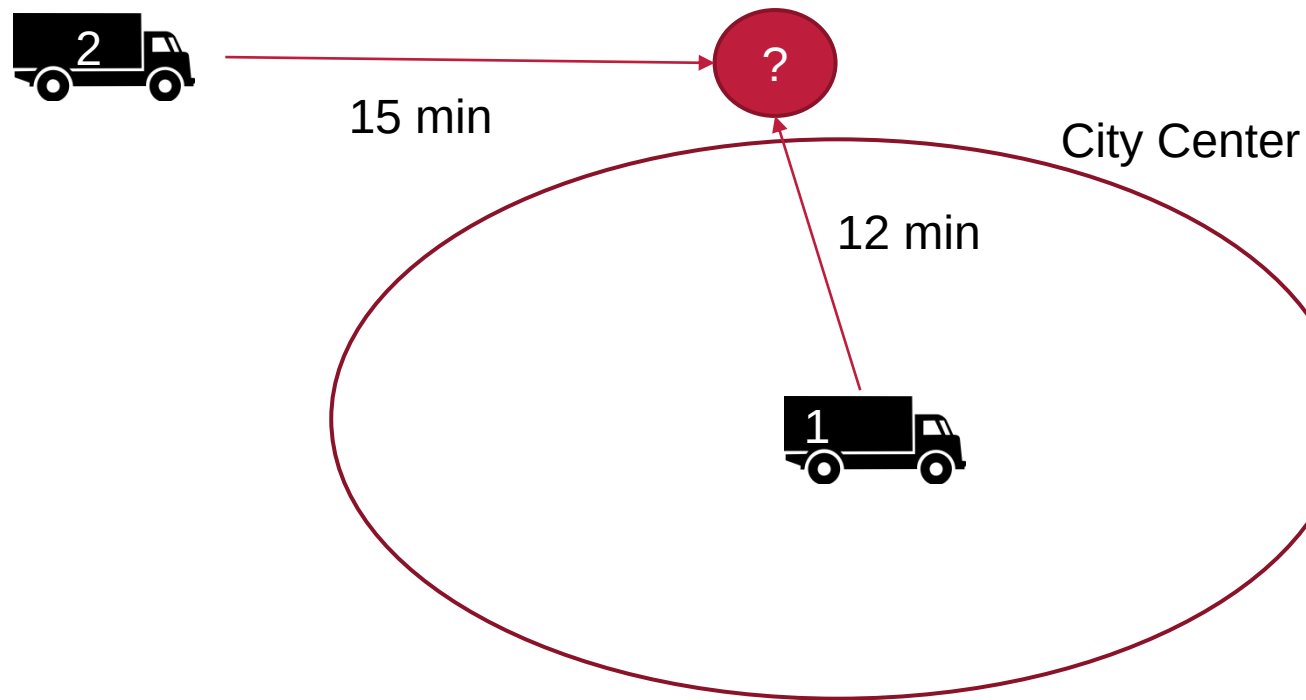
Example: Production scheduling

- Minimize delay in production scheduling
- We consider three machines M1, M2, M3
- A machine can process one job at a time
- Two jobs are to be processed on M1 → M2 → M3
- The processing time on the machines differ
- Both jobs come with a deadline of 9
- Two potential schedules are possible
- RHR is indifferent with regard to delay of schedules
- Second schedule gives more flexibility in the future



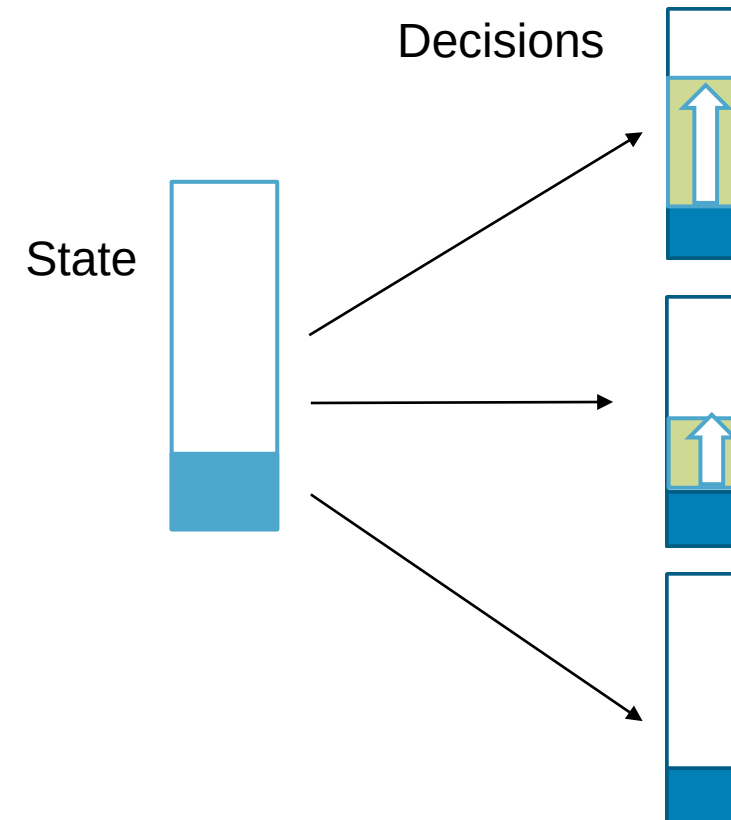
Example: Dynamic Vehicle Routing

- Objective: Minimize response time
- Two vehicles, one in city center, another vehicle far out
- RHR-decision is to send vehicle 1 ($12 \text{ min} < 15 \text{ min}$)
- No vehicles in central area with high probability of requests anymore



Example: Inventory Management

- Minimize ordering cost, inventory holding cost, and emergency replenishment cost
- Known immediate cost are for ordering and inventory holding
- Emergency replenishment cost are not known yet
- We would never order anything



Summary

- No anticipation of future changes in information and decisions
- No flexibility of solution
- Suitable for problems with mild impact of stochasticity and dynamism
- Problems where feasible decisions are already hard to find (NP-hard decision space)
- Does not work if future reward is large compared to immediate reward
- Examples:
 - Works acceptably well for scheduling with stochastic set-up times and many additional constraints
 - Works not at all for courier services (no flexibility for new requests, no coverage of service area) or inventory management (no refilling)

Agenda

- Policy Function Approximation
- Rolling Horizon Re-Optimization
- **Cost Function Approximation**
- Case Study: Restaurant Meal Delivery

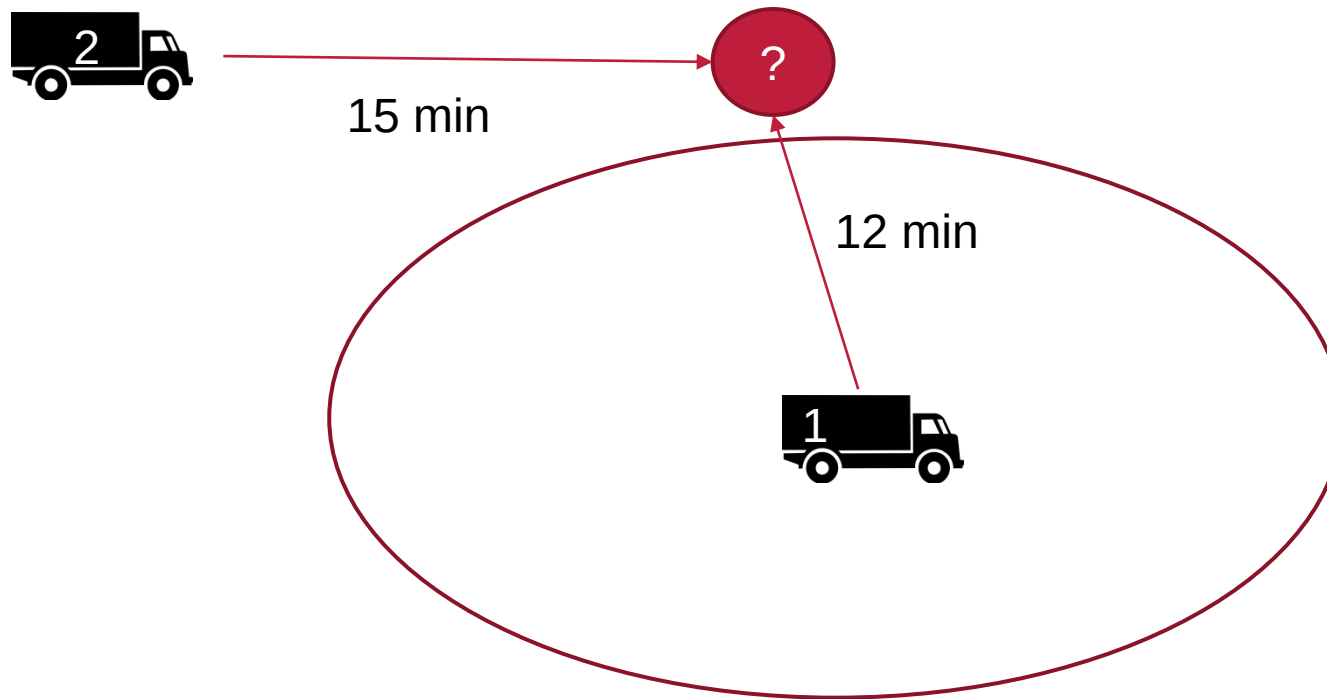


Cost Function Approximation

- Idea:
 - Incentivize (penalize) selection of “good” (“bad”) decisions by modification of reward term
 - Prohibit bad decisions by manipulating constraints
- $x = \operatorname{argmin}_{x \in X(S)} (R(S, x) + I(S, x))$
- $(A + \Theta)x \leq b + \theta$
- I depends on state and decision (can be negative)
- Some parallels to Bellman Equation (Powell 2011 denotes I as V)
- Θ, θ are usually globally set, but are also varied recently (e.g., time-dependent)
- Feasibility has to be ensured when selecting Θ, θ
- No explicit anticipation of stochasticity and future decision making.

Example: Dynamic Vehicle Routing I

- Objective: Minimize response time
- $x = \operatorname{argmin}_{x \in X(S)} (R(S, x) + I(S, x))$
- Manipulate reward function to incentivize using far-out vehicles
- $I(S, x) = -5$, if far-out vehicle is sent $\rightarrow$ tendency to utilize vehicle no. 2



Example: Production Planning

- Minimize delay (deadline given as white numbers at jobs)
- $x = \operatorname{argmin}_{x \in X(S)} (R(S, x) - I(S, x))$
- Two potential decisions x_1 and x_2 :
- x_1 :



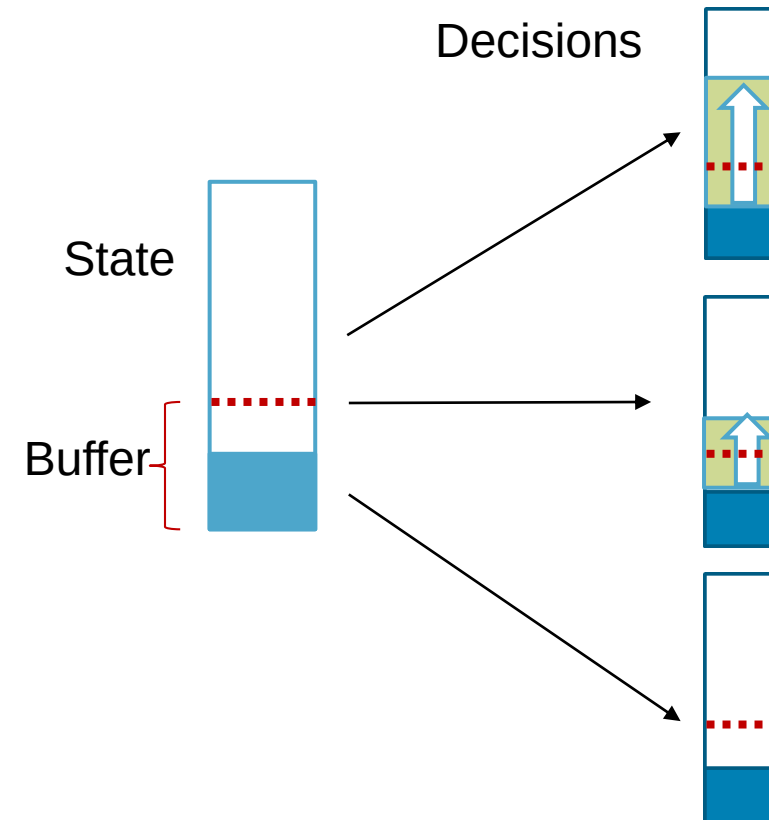
- x_2 :



- Safety buffer of 2 days: $I(S, x) = \max(0, -(\text{deadline} - \text{planned finish} - 2))$
 - $I(S, x_1) = 2$
 - $I(S, x_2) = 0$

Example: Inventory Management

- Safety buffer in constraints
- $(A + \Theta)x \leq b + \theta$
- Change in constraint:
 - “Inventory ≥ 0 ” $\rightarrow$ “Inventory $\geq \theta$ ”
- Third decision becomes infeasible



Summary

- Fast
- Easy implementation, communication, and interpretation
- Conventional MIP-solver can be used
- Very difficult to find a good tuning, especially state-dependent
- Difficult to capture long term effects of the process

From: Warren Powell <...>

I have several meta-topics that I think a young person could center an entire career around:

- o Parametric CFAs - I view this as comparable to bringing parametric statistics into optimization.
- o ...

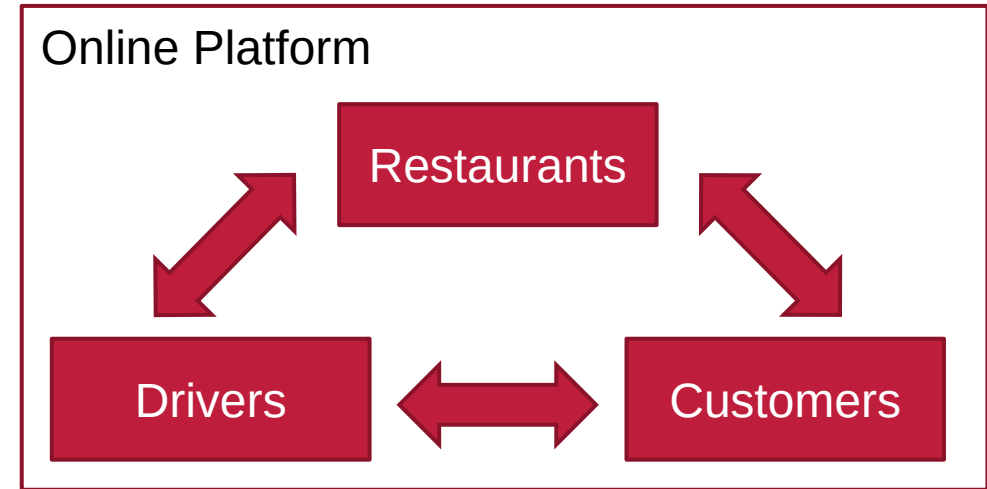
Agenda

- Policy Function Approximation
- Rolling Horizon Re-Optimization
- Cost Function Approximation
- **Case Study: Restaurant Meal Delivery**
- Ulmer et al. (2021): The Restaurant Meal Delivery Problem: Dynamic Pickup and Delivery with Deadlines and Random Ready Times, Transportation Science



Business Model

- Stakeholders:



- Objectives:

- Restaurants: Sell food, grow market through good reviews
- Customers: Delivery in time, fresh food
- Drivers: Earn money by making many deliveries
- Platform:
 - Generate revenue
 - Ensure service quality

Control of food delivery

- Observation: driver arrives late at customer
- Unreliable food preparation time at restaurant
- 1:1 assignments of orders to drivers lowers complexity → increase of costs
- Bundling multiple orders to driver → delay
- Idea: still bundle orders but include buffer in delivery time calculation (CFA)



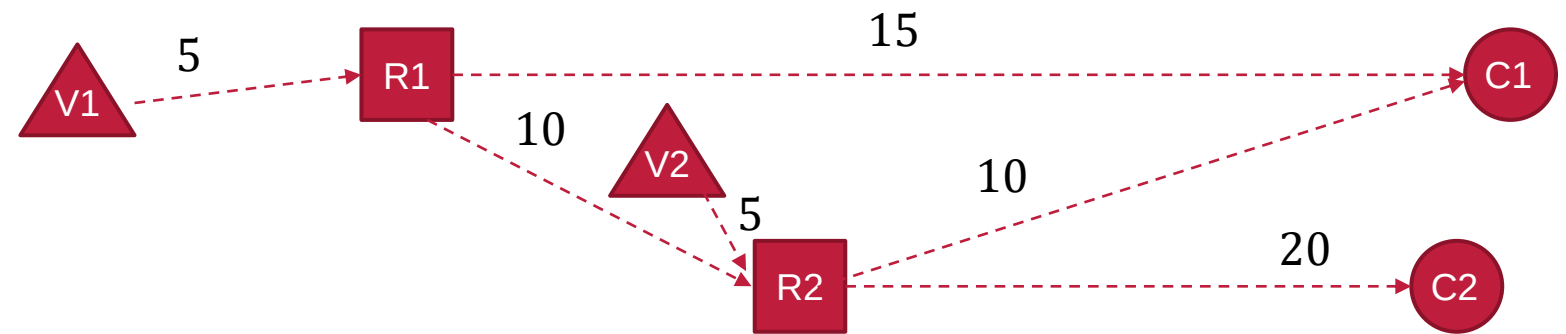
Grubhub - Grub hub sucks.

Feb 01

We ordered two meals at 10:19 pm.... delivery was supposed to be around 11:04. We didn't receive the food until almost midnight. Cold.... and the driver didn't seem to care.... it was fairly expensive food and we were really disappointed with the grub hub customer...

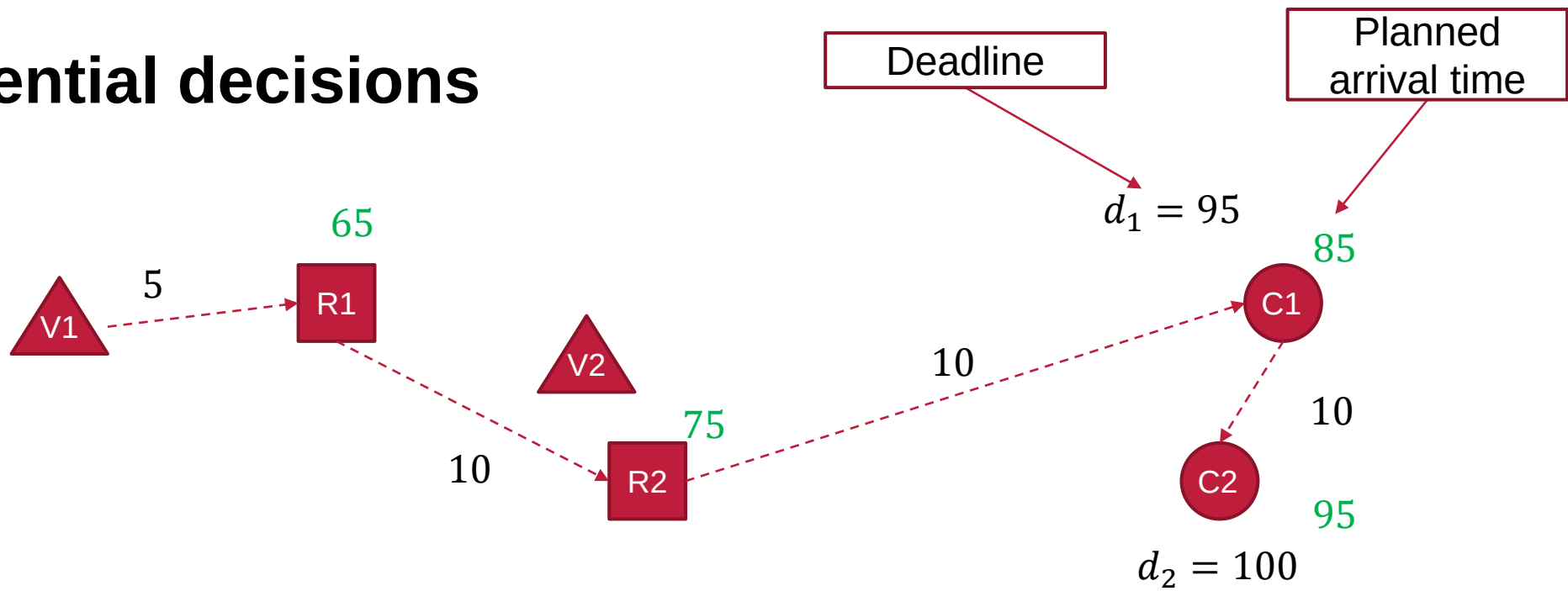
Operational control of food delivery

- Stochastically arriving orders for restaurants
- Fleet of available driver, currently located at different positions
- Stochastic food preparation times at restaurant, expected: 10 minutes
- Guaranteed delivery deadline for orders: 40 minutes
- Decisions: assignment of drivers (and routing)
- Example:
 - Customer C1 has ordered from restaurant R1 at time 55: deadline $55 + 40 = 95$
 - Current time is 60, driver V1 leaves towards R1 and will deliver to C1 at time 80
 - Also at time 60, customer C2 orders from restaurant R2: deadline $60 + 40 = 100$
 - How to proceed with respect to timely delivery? Send V1 to R2 or engage V2?

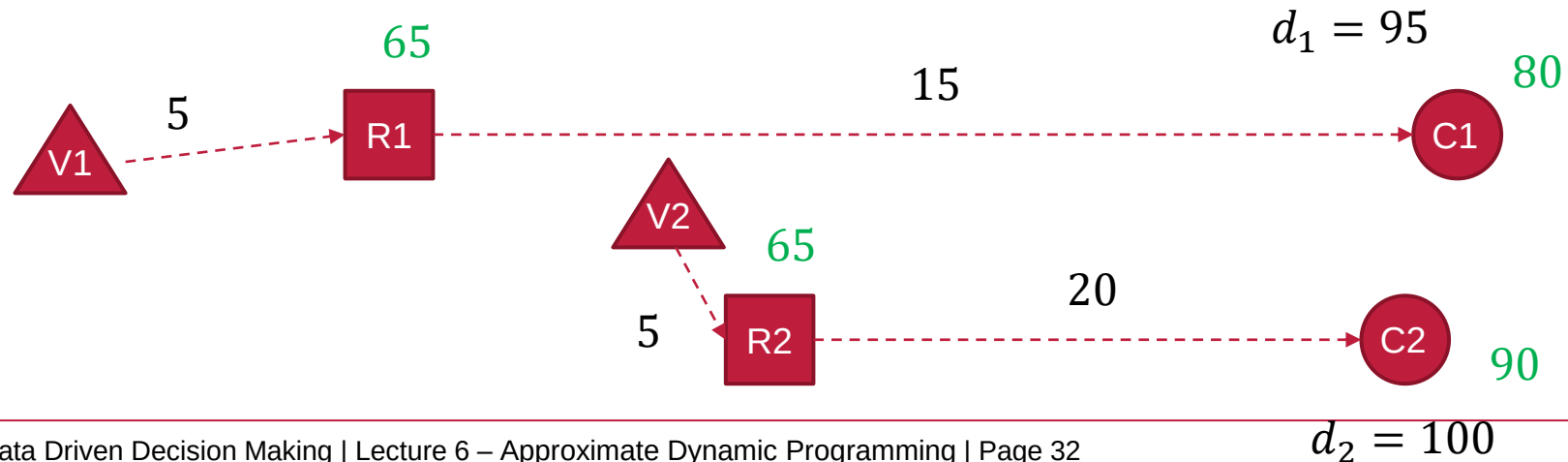


Two potential decisions

“Bundling”

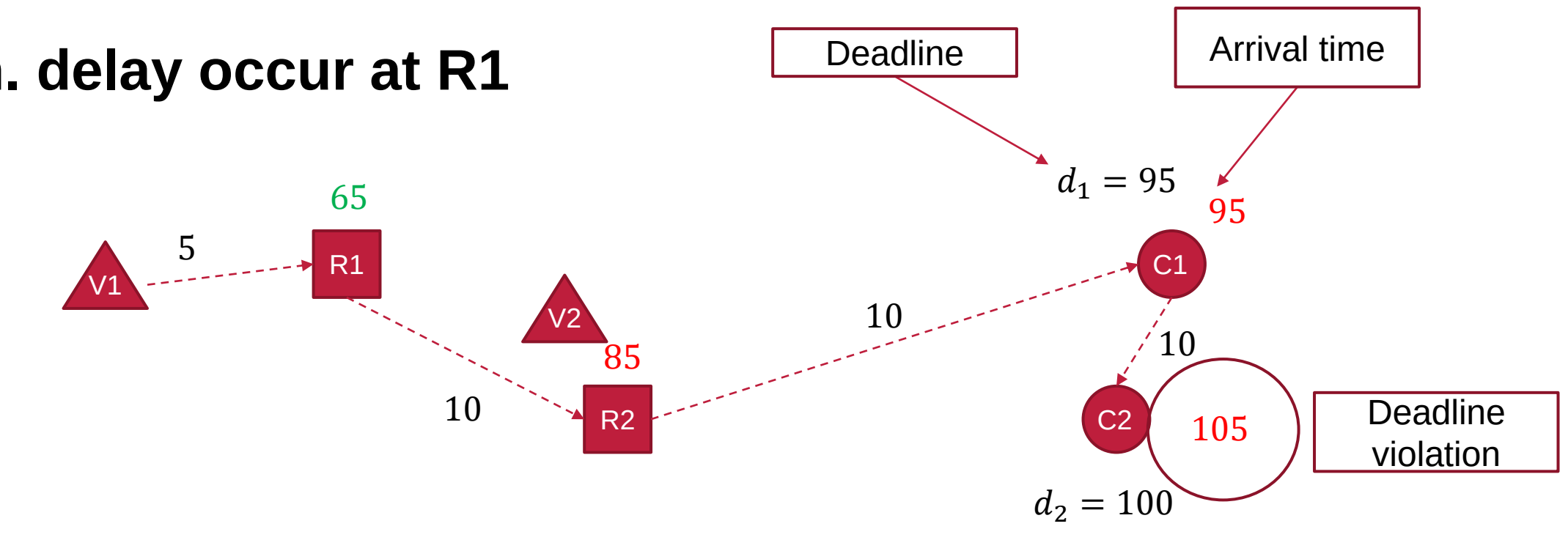


“Fastest”

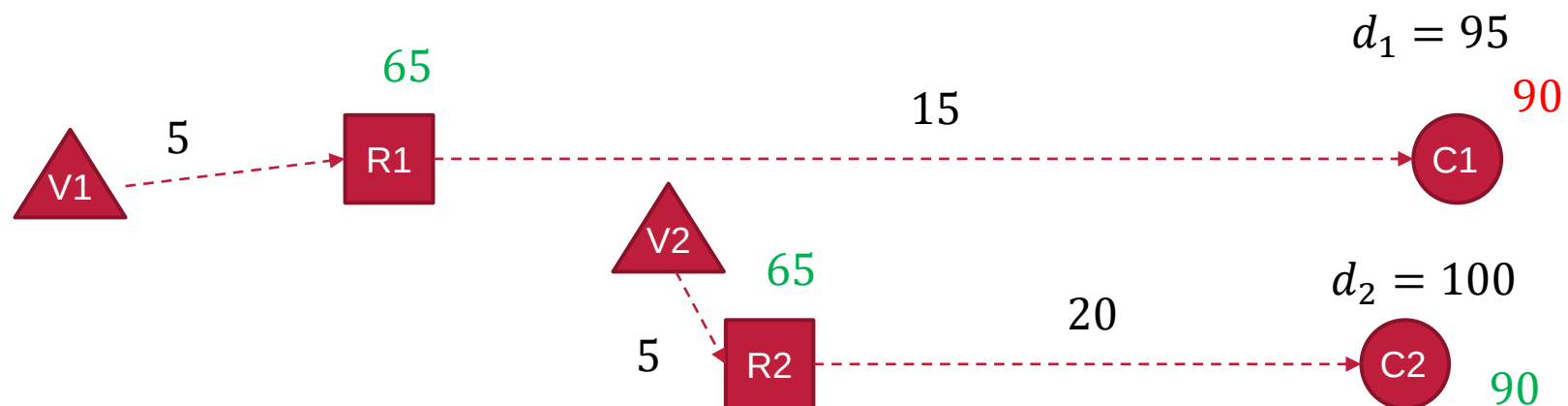


10 min. delay occur at R1

“Bundling”



“Fastest”

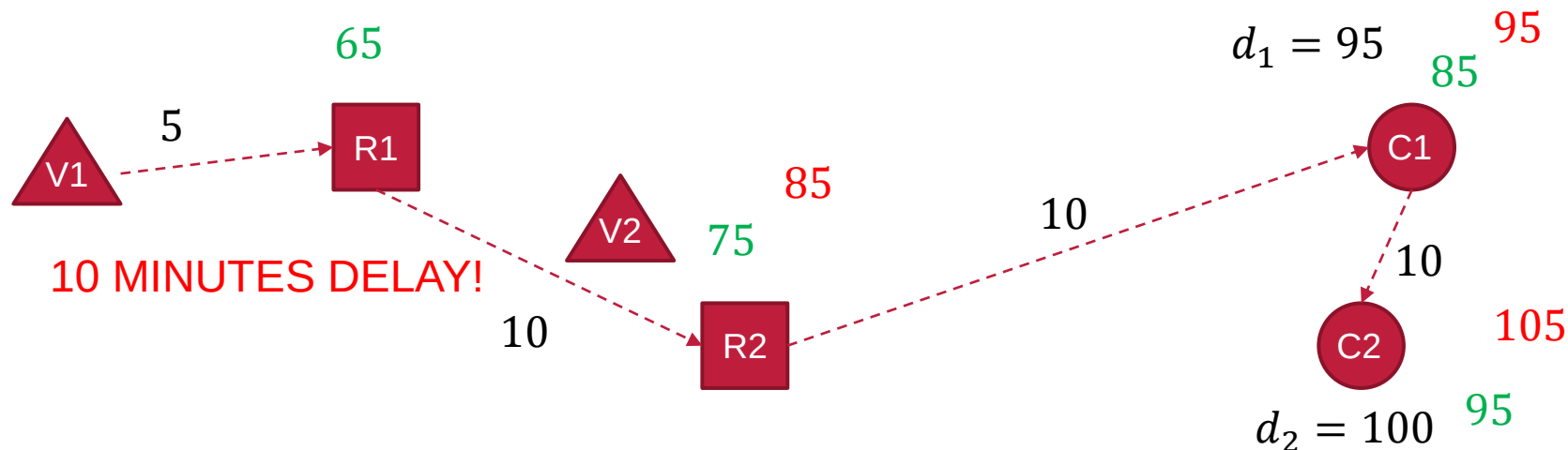


Markov Decision Process

- Decision time: Every minute
- State:
 - Point of time
 - Set of customers (Order time, restaurant, location)
 - Planned routes P : Sequence of visits and expected arrival times
- Decisions x :
 - Assignment of driver to order
 - Update of routes P to routes P^x
- Transition ω_k : occurring food preparation delay while executing plan
- Costs:
 - Delay-function $D(P) \rightarrow \mathbb{R}_+$
 - Deterministic change in delay $C^1(S_k, P, P^x) = D(P^x) - D(P) \rightarrow \mathbb{R}$
 - Stochastic difference between expected and realized delay $C^2(S_k, P^x, \omega_k) \rightarrow \mathbb{R}$

Delay-Function and Costs

- Updated V1 route: (R1,65), (R2,75), (C1,85), (C2,95)
- Delay-function: expected arrival times vs. deadlines
- Delay of V1 route: $(85 - 95)_+ + (95 - 100)_+ = 0$
- Deterministic costs $C^1(S_k, P, P^x) = 0$
- Realized stochastic costs $C^2(S_k, P^x, \omega_k) = 5$

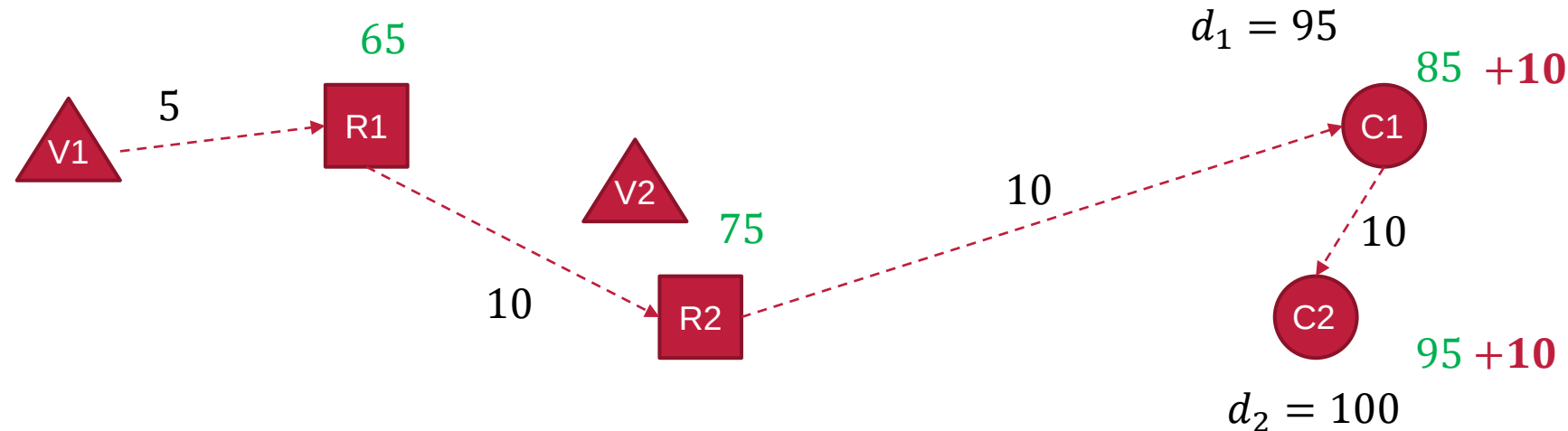


Solutions

- Bellman Equation:
$$X_k^{\pi^*}(S_k) = \arg \min_{P^x \in P(S_k)} \{C^1(S_k, P, P^x) + V(S_k^x)\}$$
- Cost Function Approximation:
 - Deterministic costs: Routing heuristic, delay first, distance second
 - Stochastic costs: Time buffers (→ Cost Function Approximation)
- Time buffer possible between extremes
 - 0 (deterministic food preparation) → bundle whenever possible
 - 40 (entire delivery time window) → always opt for fastest delivery

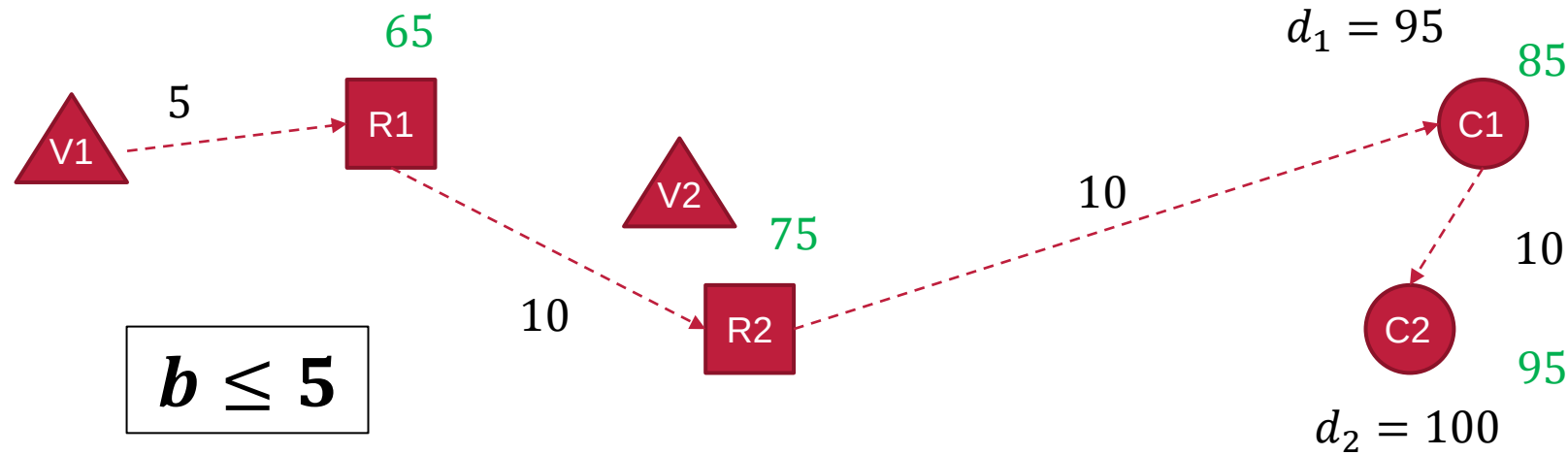
Stochastic Costs: Time Buffer (CFA)

- Motivation: Reduce “risky” arrivals close to the deadline
- Procedure: Artificially extend the arrival times at customers by a buffer value b
- Example: a buffer of 10 minutes $\rightarrow$ affects delivery > 30 minutes after order time

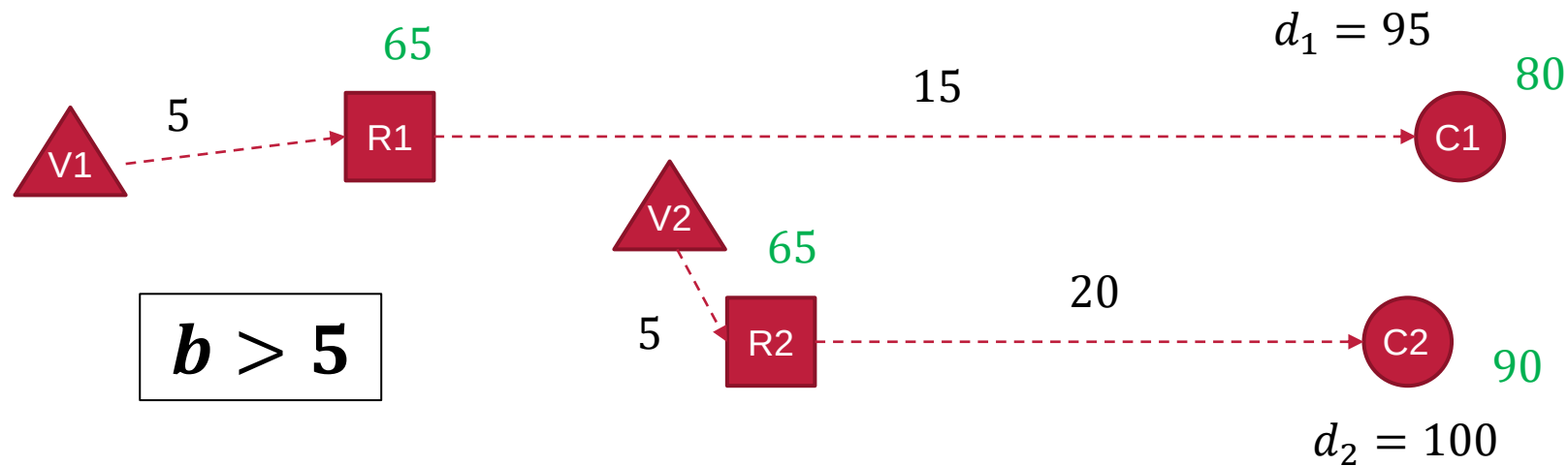


Impact of the buffer time on delivery decision

“Bundling”

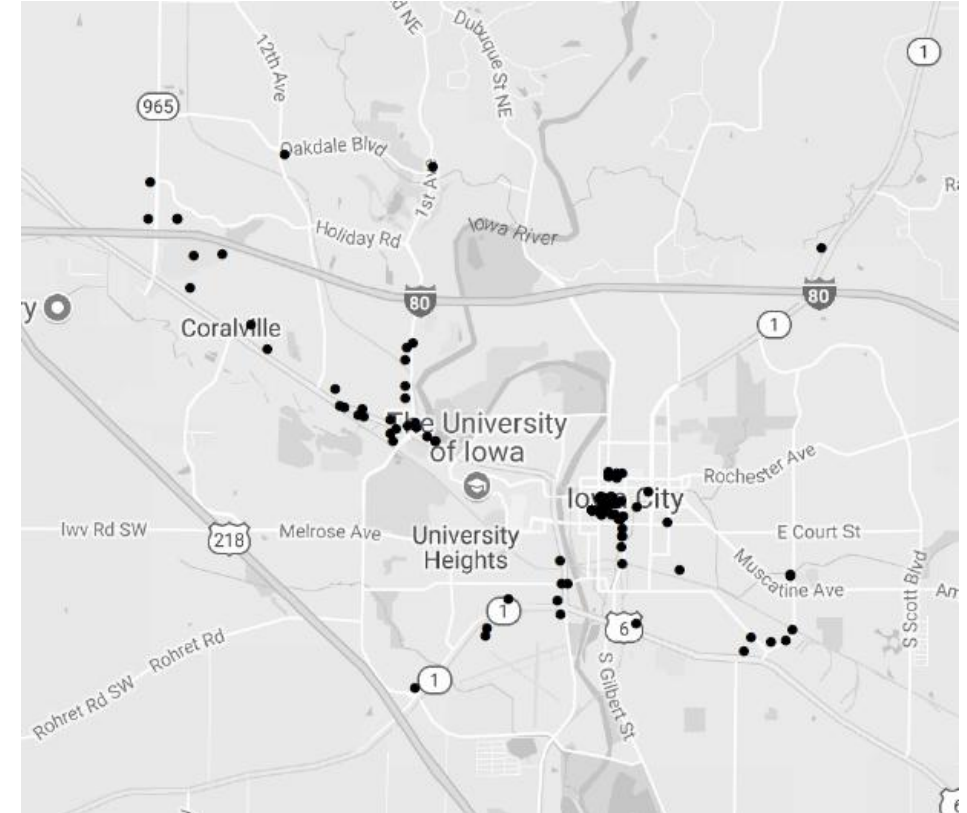


“Fastest”



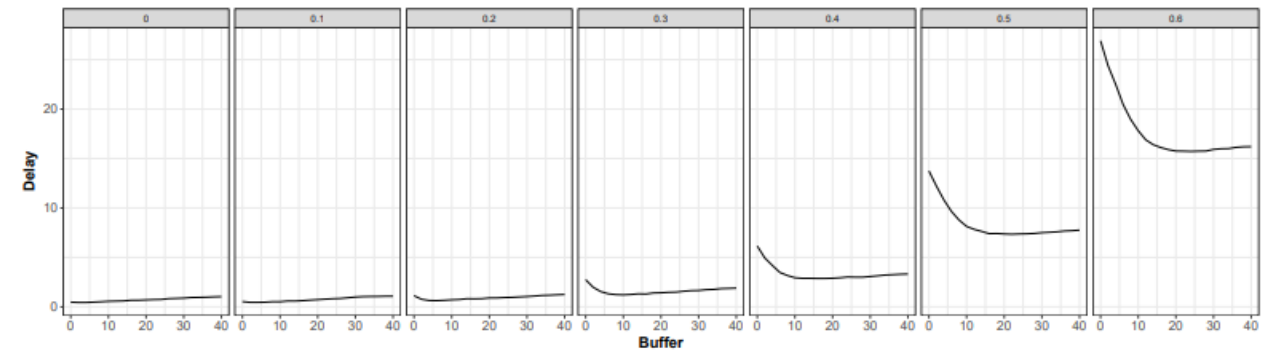
Instances and Results

- 110 restaurants
- 32000 potential customer locations
- 15 vehicles
- Orders: 180, 240, 300 per day
- Ready time follows gamma distribution with coefficient of variation (CoV): 0.0, 0.1, ..., 0.6

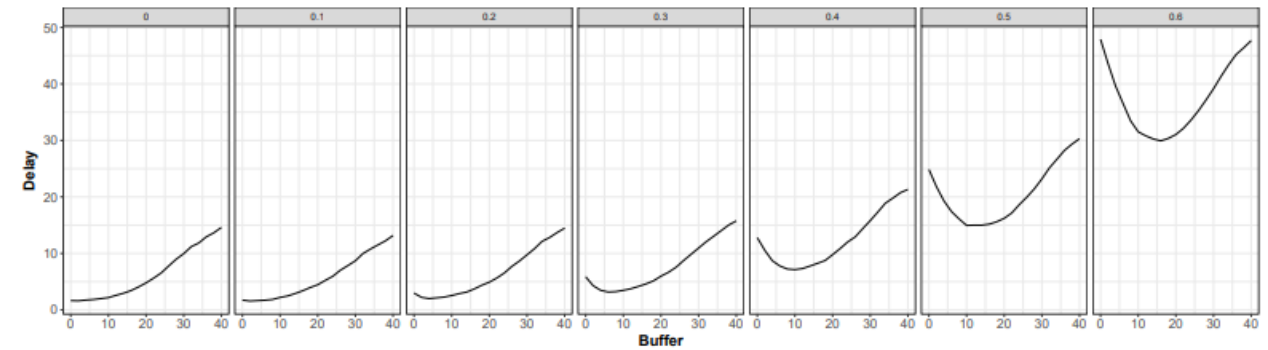


Determine buffer times

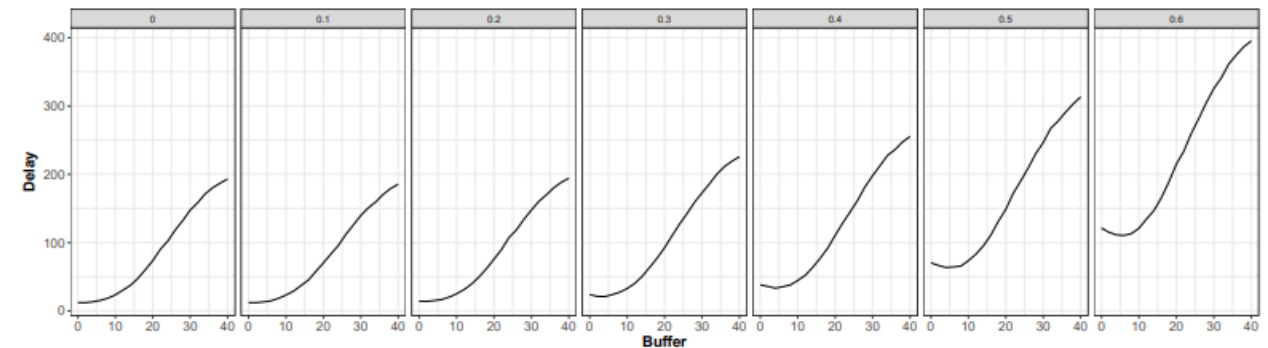
- Depict buffer time vs. observed delay
- Vary # orders and CoV of ready times
- 180 orders
 - Delay increases with increasing CoV
 - A larger buffer counteracts delay
- 300 orders
 - Buffer times actually increases delay
 - Consolidation of orders due to bundling is required to cope with #orders
- 240 orders
 - There is an „optimal“ buffer time for CoV



(a) 180 Expected Orders



(b) 240 Expected Orders



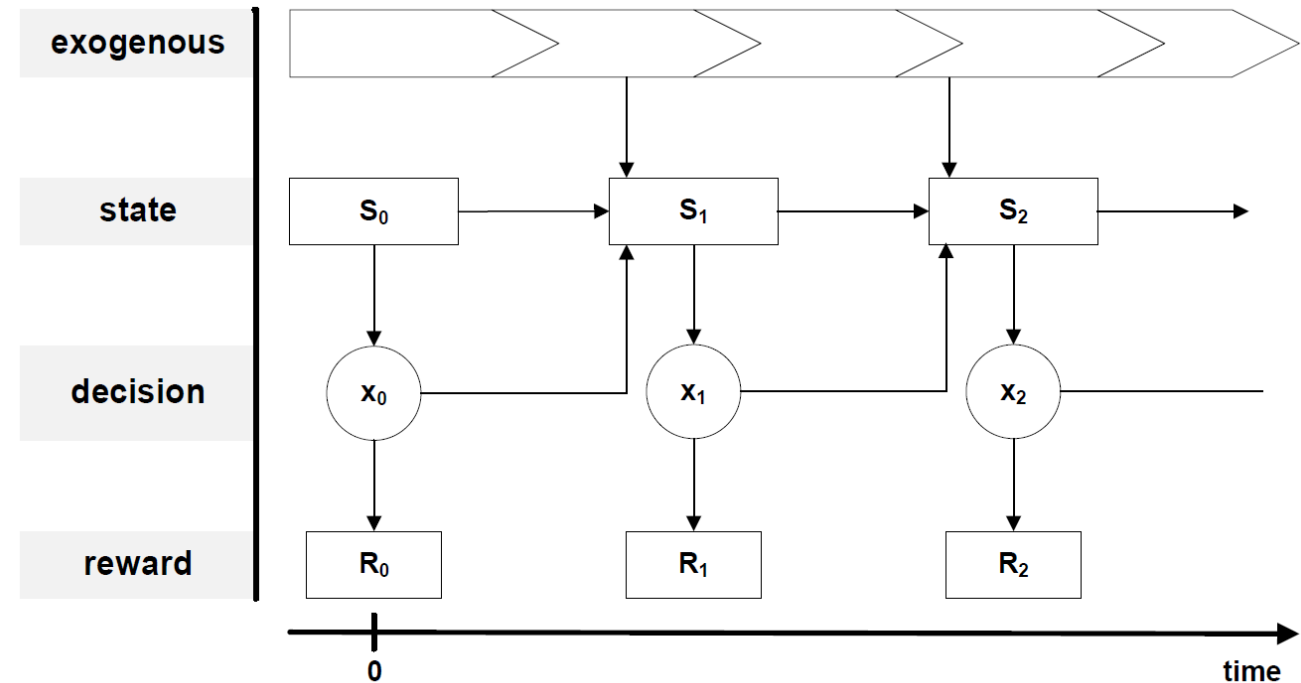
(c) 300 Expected Orders

Parametrizable CFA: State-Dependent Buffers

- Different states might require different buffers (current and expected orders, restaurant workload, vehicle positions, etc.)
- First step: Time-dependent buffers, one buffer per hour
- Interdependencies between buffers over time: Reinforcement learning
- Additional delay reductions of about 6% possible
- Open research question: include more state detail

Summary

- PFA: Derive a policy based on a rule-of-thumb (or analytically)
- CFA: Manipulate reward function or constraints
- Both usually improve simple Rolling Horizon Re-Optimization
- None of them considers future information model realizations and decision model instances explicitly



Outlook Policy Classes (Compare Powell 2011)

- Policy Function Approximation
- Rolling Horizon Re-Optimization
- Cost Function Approximation
- **Look-ahead Models**
- Value Function Approximation

Rule-Based

Optimization-Based